

Topology-Aware Sample Sort

CMSC 714 Project — The bisection-bandwidth axis of the topology study

Sonok Mahapatra

Group: J. LaGattuta, C. Dalziel, M. Wijitharathna, S. Mahapatra

Spring 2026

Project Context

Group study: how do four MPI workloads perform on five simulated network topologies? Built under SimGrid 4.1 SMPI; topology defined by XML.

Workloads

- 3D stencil
- 3D FFT
- Barnes–Hut N-body
- **Sample sort (mine)**

Topologies

- ring
- 3D torus
- hypercube
- fat tree
- dragonfly

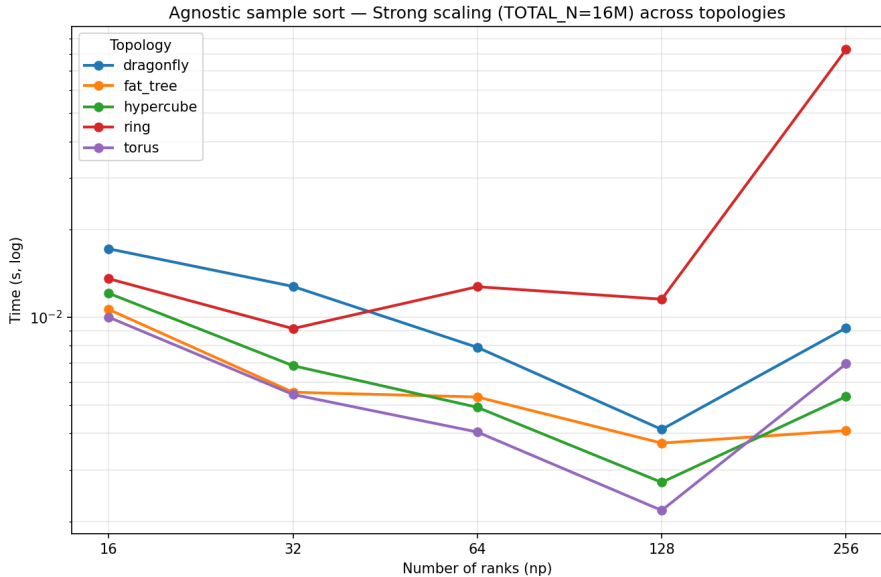
This slot: the sample-sort row of the matrix. Sample sort is fundamentally a giant *all-to-all* workload, so it exercises the network's **bisection-bandwidth axis** more than any other algorithm in the study.

Sample Sort (PSRS) — 8 Steps, 1 Bottleneck

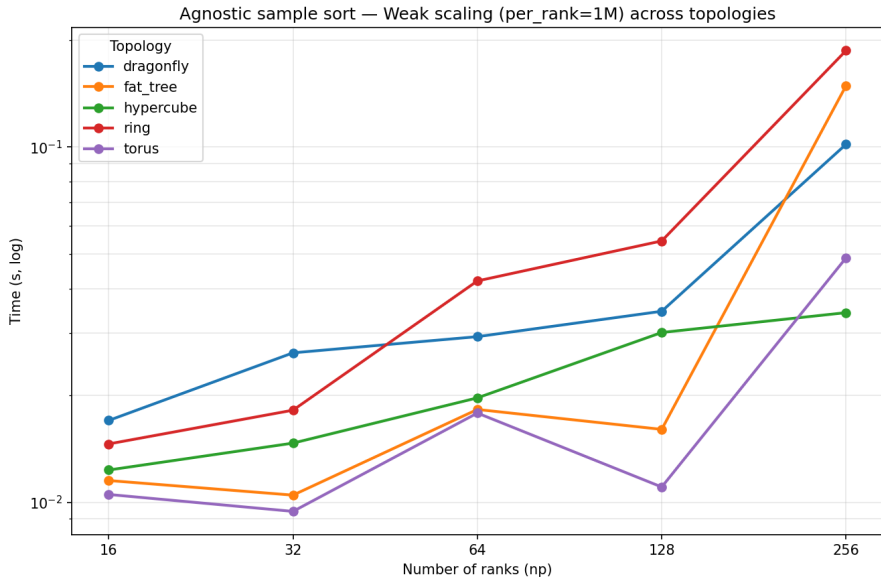
- 1 Local sort (N/p keys per rank).
- 2 Pick $p - 1$ regular samples per rank.
- 3 `MPI_Allgather` samples (every rank holds $p \cdot (p - 1)$ samples).
- 4 Sort gathered samples; pick $p - 1$ global splitters.
- 5 Partition local data by splitters $\rightarrow$ `send_counts[p]`.
- 6 `MPI_Alltoall` to exchange the counts.
- 7 `MPI_Alltoallv` to redistribute data $\leftarrow$ topology-sensitive.
- 8 Local sort of received bucket.

Why this is a topology test. Step 7 is one global all-to-all over ~ 8 MB-per-rank. Bisection bandwidth ($O(1)$ on ring, $O(p)$ on fat tree) sets a hard ceiling on how fast it can possibly run.

Agnostic Baseline — Strong Scaling (TOTAL_N=16M)



Agnostic Baseline — Weak Scaling (1M keys/rank)



Topology-Aware Variants — One Per Topology

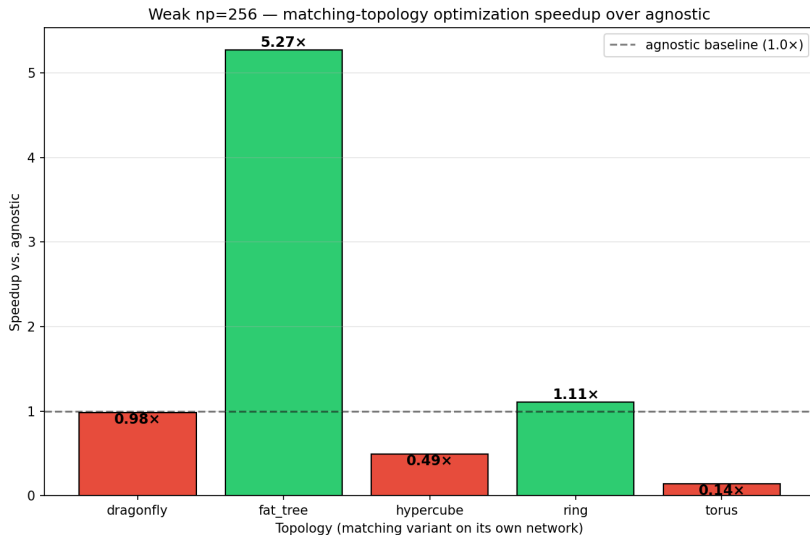
Step 7 was replaced with a topology-aware redistribution in 5 separate binaries:

Variant	Step 7 algorithm
ring	$p - 1$ pairwise MPI_Sendrecv at distances $1..p - 1$
torus	2-phase row $\rightarrow$ column Alltoallv on Cartesian sub-comms
hypercube	$\log_2 p$ rounds of recursive halving with per-bucket forwarding
fat_tree	Chunked Alltoallv (4 sub-rounds, smaller in-flight buffers)
dragonfly	2-phase intra-group $\rightarrow$ inter-group Alltoallv

All 5 variants pass `--verify` on $np \in \{16, 32, 64, 128, 256\}$.

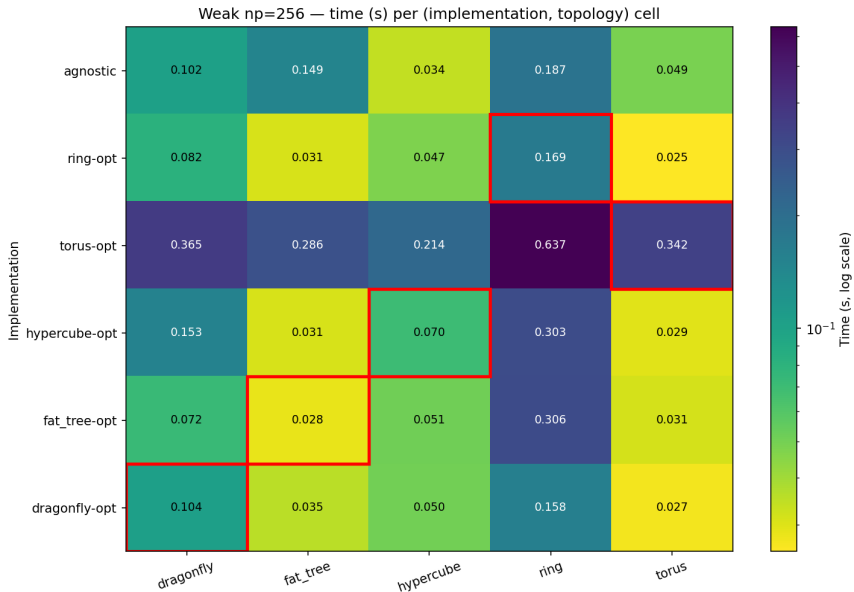
Full grid: 6 binaries (5 optimized + agnostic) $\times$ 5 topology XMLs $\times$ 5 np $\times$ 2 modes = **300** smpirun runs.

Optimization Results — Diagonal Speedup, Weak np=256



The story: only **fat_tree-opt** and **ring-opt** actually help. **torus-opt** is 14× slower than agnostic on its own 7 / 11

Optimization Results — Full 6×5 Grid (Weak np=256)



Three Findings

A. Fat-tree chunking solved the np=256 weak surprise.

The mystery slow-down had a measurable cause (in-flight buffer pressure on the simulated links). Splitting the all-to-all into 4 sub-rounds dropped it from 0.149 s to 0.028 s — **5.27× speedup**.

B. Naive 2-phase decomposition is a trap.

The textbook “row → column on a 2D grid” optimization (torus-opt) is **14× slower** than agnostic at strong np=256. SMPI’s default `coll-selector:ompi` is already smart; the explicit sub-comm setup + double-packing overhead dwarfs the savings.

C. Algorithmic simplicity beats topology-matching.

Fewer rounds wins, even on the “wrong” topology. hypercube-opt on a *ring* (strong np=256) is 0.055 s vs. agnostic 0.083 s — 1.5× faster on a topology it wasn’t designed for.

For the team's broader question (which topologies suit which algorithms?):

- Sample sort cleanly exposes the **bisection-bandwidth axis**: the agnostic ranking ($\text{fat_tree} \approx \text{hypercube} \approx \text{dragonfly} \ll \text{torus} \ll \text{ring}$) is the answer for an algorithm dominated by global all-to-all.
- Topology-matched code mostly *preserves* that ranking; it does not unlock dramatically new ones.

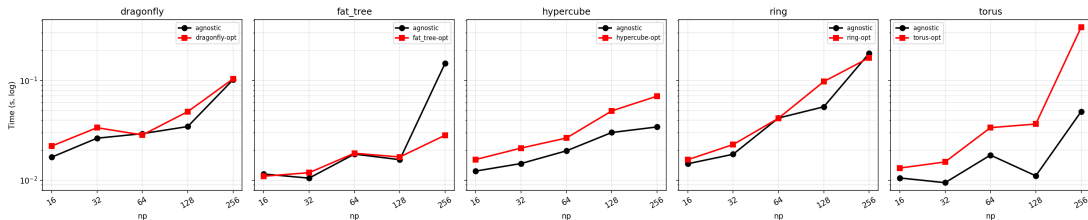
The nuanced result of this slice:

Topology-aware code only wins when it targets a specific measured bottleneck — not when it merely “matches” the network structurally.

That is more honest than “topology-aware is always better” and more useful than “topology-aware never helps.”

Backup — Per-topology Comparison

Weak scaling — agnostic vs. matching-topology optimization, per topology



Agnostic (black, circles) vs. matching-topology optimization (red, squares) across all np, weak scaling. Fat tree is the only big visible win.